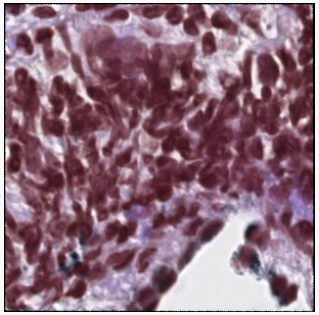
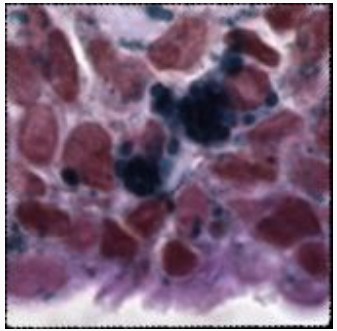
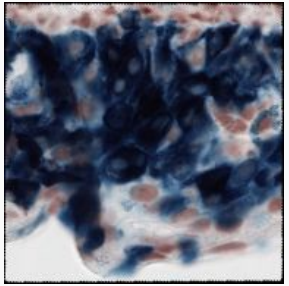
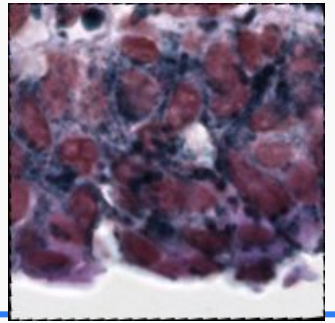
Team 5
Presentation 1

Andy Fonseca, Javier Hernandez, Lidia Mora Gallegos, Lucas McDonough

Engineering Images

3 Cases (35, 62, 102)

- 15/21 slices. (Sent 6 to Krish, established we need more information on how to annotate these.)
- Created 2,362 patches
  - All which have been uploaded too,
    `/projects/e32998/patches_cases_8_20_33_35_62_102/.`
- Patches Created

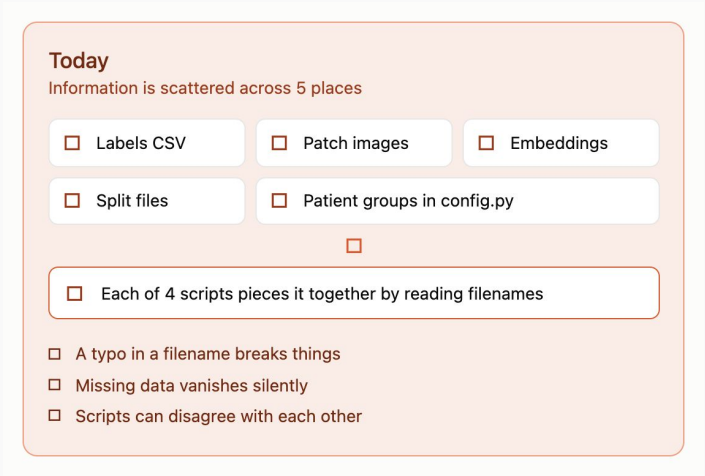





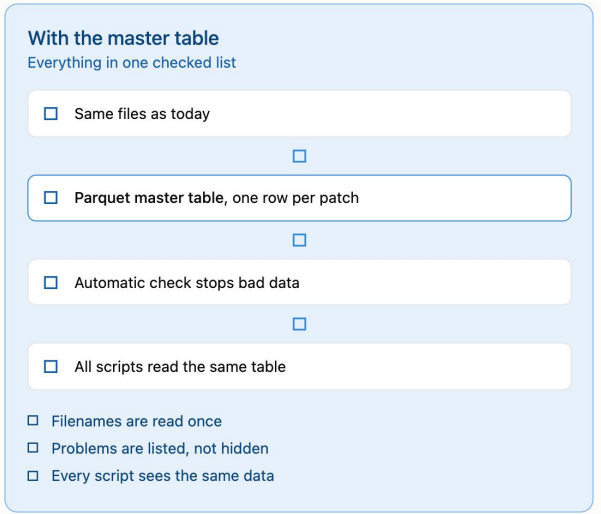
Javier

Engineer Structure

Next Steps:

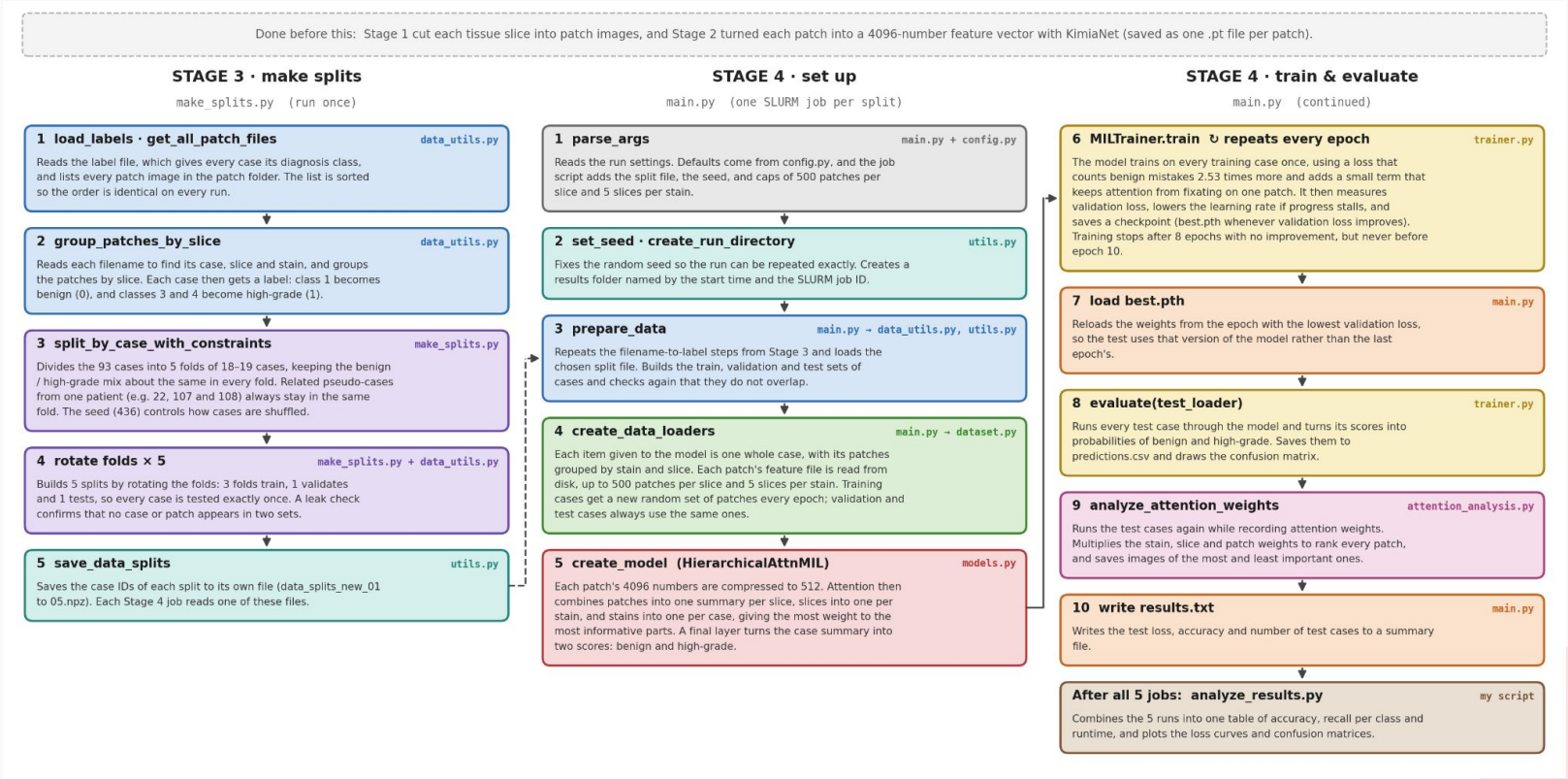
1. Versioned Dataset:
   a. Adding new samples to dataset we are currently working on
      i. Once we get green light from Krish
2. Building Metadata table
   a. A master Parquet table of all our image patches, plus an automatic check that catches data problems before training.





Javier

Code Flowchart



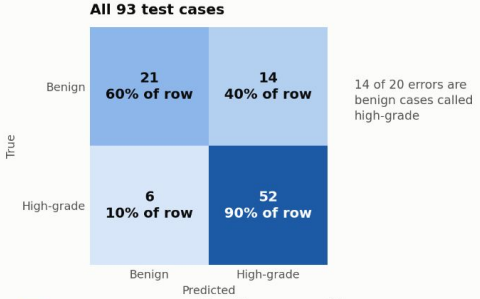
Lucas

Results

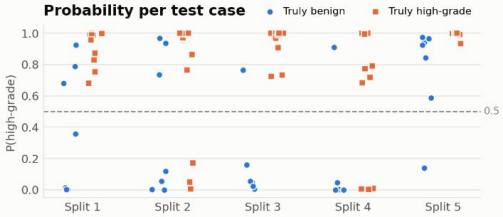
Seed 436 · 93 cases, each tested exactly once · test set scored with best-validation weights

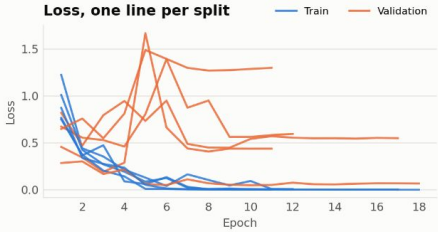
Mean accuracy	High-grade recall	Benign recall	Runtime per split
0.78	**0.90**	**0.60**	**18 min**
± 0.12 across splits (range 0.67–0.95)	share of high-grade cases caught	share of benign cases called benign	range 13–26 min, CPU only

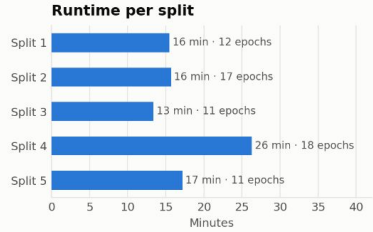




14 of 20 errors are benign cases called high-grade







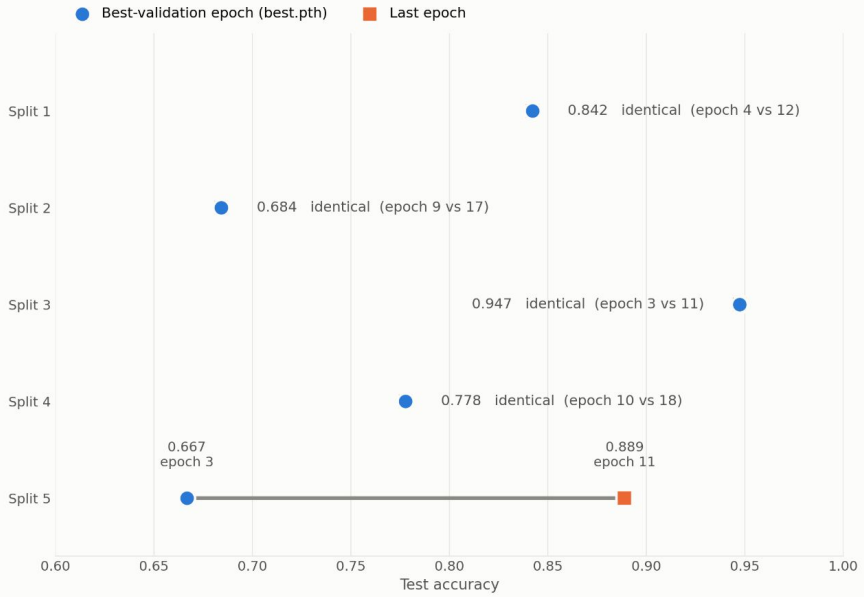
Wrong answers are still confident: median 0.93

Train loss reaches ~0 within ~5 epochs; validation does not → overfitting

Lucas

Adjusting the Epoch

Same test sets, same trained models — only the saved checkpoint that gets scored is different



Mean test accuracy

0.784 best.pth

0.828 last epoch

What I changed

main.py now reloads best.pth (lowest validation loss) before testing, instead of testing the last epoch's weights.

What I found

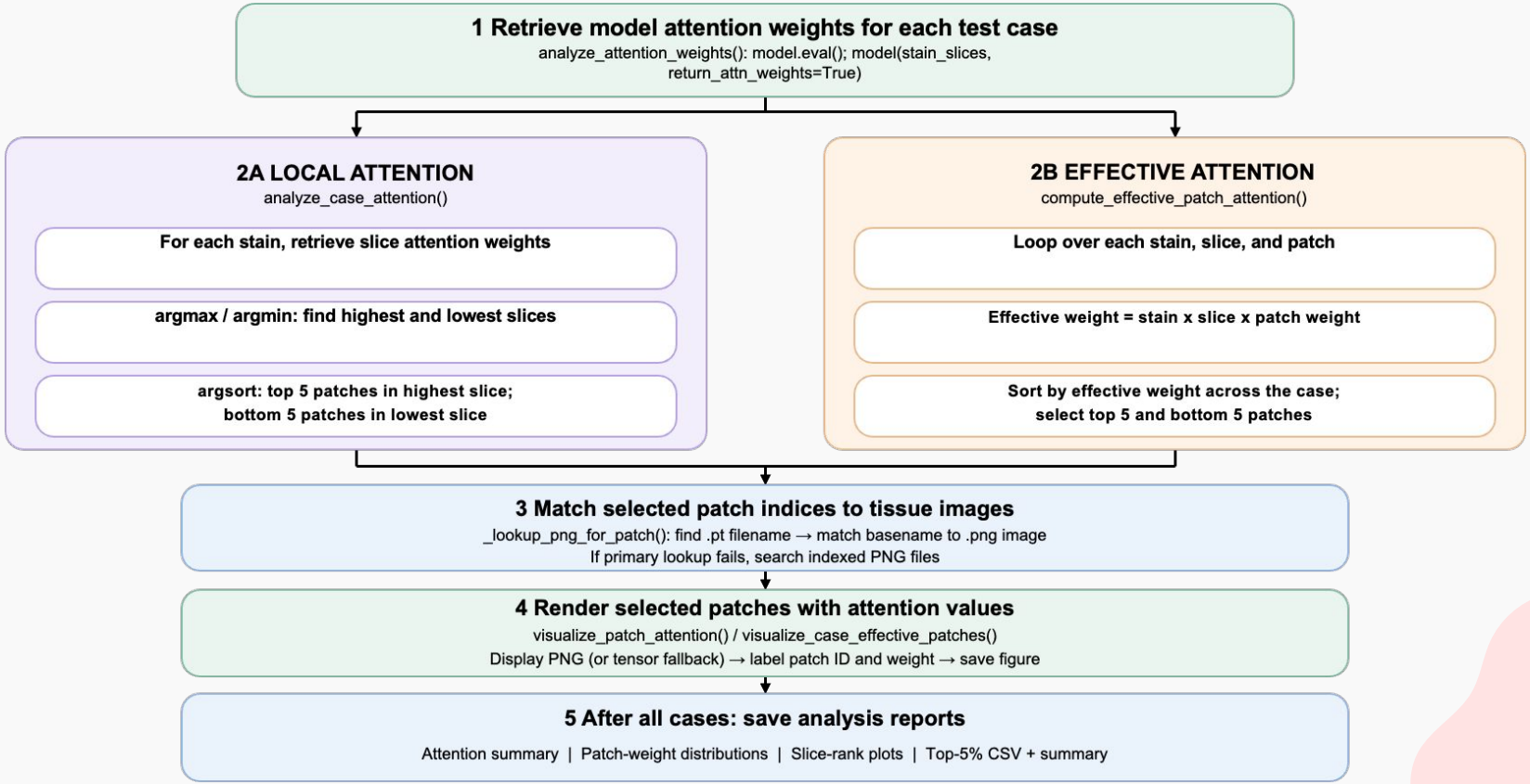
4 splits give the same accuracy. In split 5, the 'best' epoch 3 scores 0.667, while the last epoch scores 0.889.

Why

The validation set has only 18–19 cases, so its loss is too noisy to pick the best epoch reliably. Early stopping depends on the same noisy number.

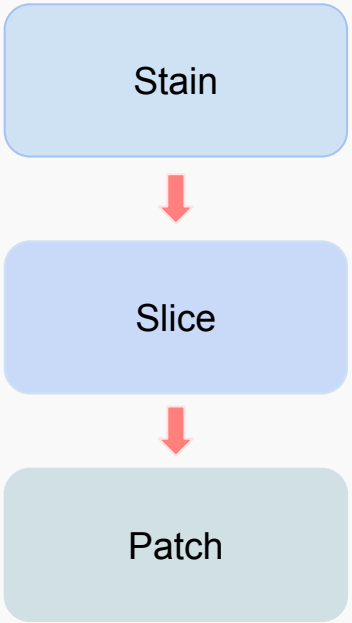
Lucas

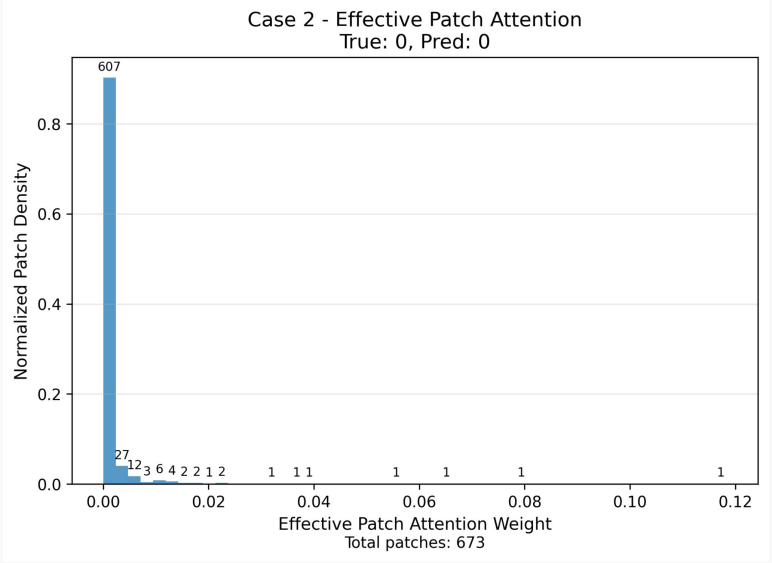
Analysis Flowchart



Visualizing Attention in MIL

Attention Hierarchy Diagram





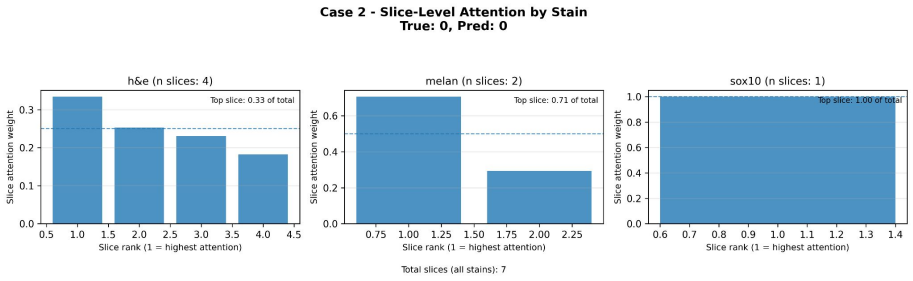
Attention is concentrated in a small number of patches, while most receive very low effective attention weights.

Effective patch attention = stain weight x slice weight x patch weight

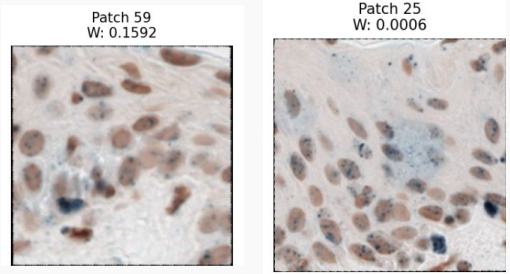
Correct Predictions

Correctly Classified Benign - Case 2

True: Benign | Predicted: Benign | 99.97% P(Benign)



Stain Attention: Sox10 50.46% | Melan 49.33% | H&E 0.21%

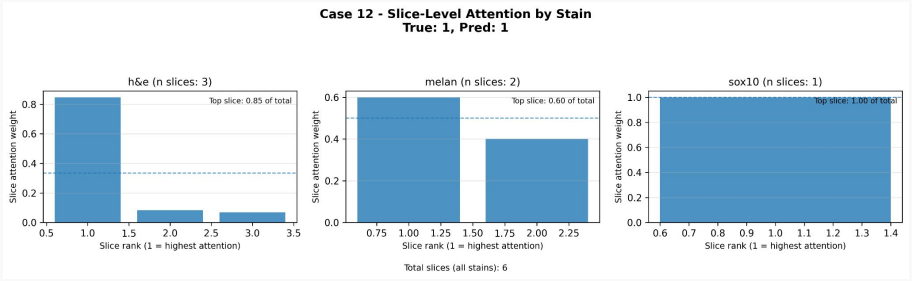


High Sox10 Patch Attention Weight

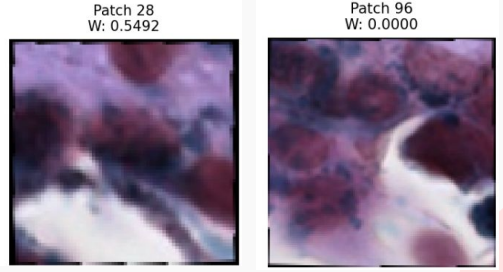
Low Sox10 Patch Attention Weight

Correctly Classified High-Grade - Case 12

True: High-Grade | Predicted: High-Grade | 99.65% P(high-grade)



Stain Attention: H&E 51.36% | Melan 33.76% | Sox10 14.88%



High H&E Patch Attention Weight

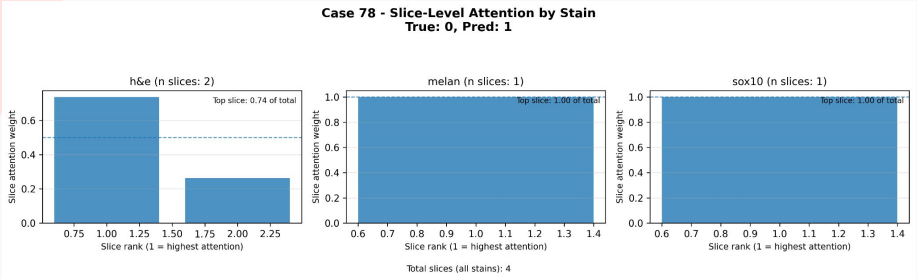
Low H&E Patch Attention Weight

Lidia

Incorrect Predictions

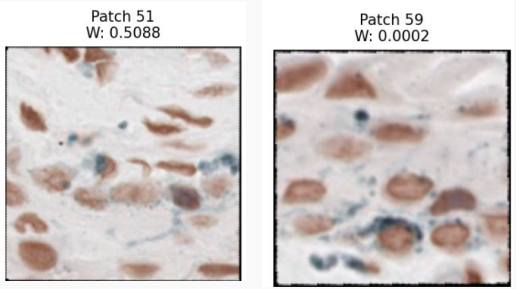
False Positive - Case 78

True: Benign | Predicted: High-grade | 75.87% P(high-grade)



Stain Attention: **Sox10 76.18%** | Melan 23.67% | H&E 0.15%

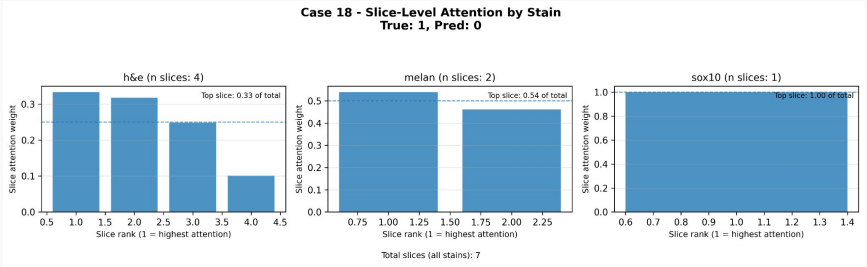
High Sox10 Patch Attention Weight



Low Sox10 Patch Attention Weight

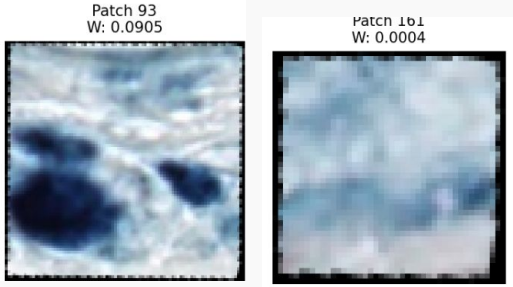
False Negative - Case 18

True: High-Grade | Predicted: Benign | 97.64% P(benign)



Stain Attention: **Sox10 66.63%** | Melan 32.85% | H&E 0.53%

High Sox10 Patch Attention Weight



Low Sox10 Patch Attention Weight

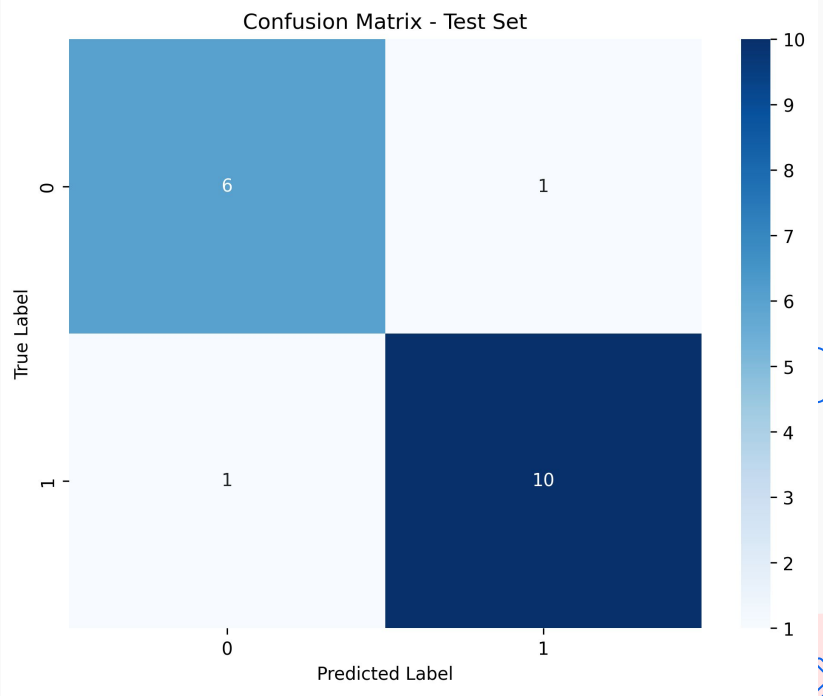
Lidia

Findings and Limitations

Test Performance

88.89% Accuracy

16 out of 18 cases correct



Key Findings

- Attention varied across stains and cases
- Both incorrect predictions emphasized Sox10

Limitations:

- Small test set of 18 cases
- Attention does not necessarily indicate clinical importance